Periodic Neighbor Search Specification

Production Dense, Cell-List, and Verlet Execution Policy for mdstats

mdstats

2026-07-14

Contents

Document contract

This document is the normative stage S4 specification for exact periodic neighbor-search execution in `mdstats` version `0.14.1`.

It is written for:

1. scientific and implementation review by humans;
2. AI contextualization before later maintenance or extension work.

The Markdown and PDF versions contain the same normative content. Source code remains authoritative if a documentation defect is found.

Primary implementation files:

```
mdstats/analysis/_neighbors.py
mdstats/analysis/_cell_list.py
mdstats/analysis/_verlet_cache.py
mdstats/analysis/neighbor_search.py
```

Current consumers:

```
mdstats/analysis/rdf.py
mdstats/analysis/coordination.py
mdstats/analysis/bond_angle.py
mdstats/analysis/atomic_connectivity.py
```

The public policy objects are importable from `mdstats`:

```
from mdstats import (
    CellListOptions,
    NeighborCacheIntervalStatistics,
```

```
    NeighborCacheStatistics,
    NeighborListResult,
    NeighborSearchDiagnostics,
    NeighborSearchOptions,
    NeighborSearchSession,
    VerletCacheOptions,
    VerletPairCache,
)
```

`PairCounting`, `NeighborSearchBackend`, and the low-level stateless builder are internal execution interfaces. Scientific modules should normally expose `NeighborSearchOptions` rather than these internals.

Motivation

RDF, coordination, bond-angle, and distance-connectivity calculations repeatedly solve the same geometric problem: enumerate selected atom pairs within a strict periodic cutoff. A blocked dense search is simple and exact, but its pair work is quadratic. An exact triclinic cell list reduces candidate work for larger systems. A Verlet cache further avoids rebuilding the candidate list on every trajectory frame.

The S4 subsystem centralizes these choices so that scientific modules do not own backend logic or cache state. Its goals are:

- preserve one exact scientific neighbor contract;
- select a conservative backend automatically when requested;
- reuse candidates only under a proven completeness bound;
- retain deterministic results and auditable provenance;
- preserve explicit dense and cell-list overrides;
- allow permanent dense fallback when an optimized path is unsupported or too complex.

Optimization must not change a graph, histogram, angle, coordination number, or normalization.

Scope and ownership

The subsystem owns:

- minimum-image pair geometry;
- strict cutoff filtering;
- dense and exact cell-list execution;
- cell-list lattice reduction and metric-stencil construction;
- request normalization and request digests;
- fixed-cell and deformation-aware Verlet validity;
- automatic backend policy;
- cache and backend diagnostics;
- exact fallback behavior.

Scientific consumers own:

- atom and frame selections;
- cutoff meaning and provenance;
- RDF normalization;
- coordination statistics;
- angle construction and weighting;
- hysteretic or reference bond state;
- graph cataloging and higher topology.

A consumer may request neighbors many times, but it must not reproduce cache validity tests or infer backend-specific scientific behavior.

Coordinate, unit, and periodic conventions

Row-vector cell convention

The cell matrix contains lattice vectors as rows:

$$H = \begin{pmatrix} \mathbf{a}^T \\ \mathbf{b}^T \\ \mathbf{c}^T \end{pmatrix}.$$

Fractional row vectors map to Cartesian coordinates by

$$\mathbf{r}_i = \mathbf{s}_i H.$$

All cells, positions, displacement vectors, cutoffs, and skins are in angstrom. No unit conversion occurs inside the neighbor subsystem.

Minimum-image vector and image shift

For center atom i and candidate atom j, the returned vector is

$$\mathbf{d}_{ij} = \mathbf{r}_j + \mathbf{n}_{ij} H - \mathbf{r}_i,$$

where $\mathbf{n}_{ij} \in \mathbb{Z}^3$ is zero along nonperiodic axes and is chosen to minimize $\|\mathbf{d}_{ij}\|$ under the supported unique-image regime. `image_shifts[k]` stores $\mathbf{n}_{ij}$ for returned pair `k`.

The scientific inclusion rule is always

$$\|\mathbf{d}_{ij}\| < r_{ij}^{\mathrm{cut}}.$$

The inequality is strict. A pair exactly on the cutoff is excluded.

Unique-image regime

The physical or list cutoff must remain below the shortest ambiguous periodic translation radius for every selected frame. For a fully periodic lattice this is one half of the shortest nonzero lattice-vector norm:

$$r_{\mathrm{safe}} = \frac{1}{2} \min_{\mathbf{m} \in \mathbb{Z}^3 \setminus \{\mathbf{0}\}} \|\mathbf{m} H\|.$$

For partial periodicity, only translations on periodic axes participate. The first production subsystem does not return multiple images of one atom pair.

Scientific neighbor contract

A valid `NeighborListResult` is deterministic and CSR grouped by the ordered center selection. It preserves:

- the requested center order;
- canonical pair filtering;
- one minimum-image vector and image shift per retained pair;
- strict cutoff inclusion;
- no self pair;
- no duplicate pair in unordered-identical mode;
- stable row and within-row ordering;
- read-only result arrays.

Two exact backends are scientifically equivalent only when all of the following match:

```
center_indices
neighbor_indices
offsets
vectors
distances
image_shifts
```

`cutoff`
`pair_counting`

The `backend` field is provenance and may differ.

Data structures

`PairCounting`

```
class PairCounting(str, Enum):
    DIRECTED = "directed"
    UNORDERED_IDENTICAL = "unordered_identical"
```

`DIRECTED` evaluates neighbors for every center row. Center and candidate selections may be disjoint or identical according to the low-level validation contract.

`UNORDERED_IDENTICAL` is valid only when center and candidate selections are exactly identical. Each unordered pair is retained once.

`NeighborSearchBackend`

```
class NeighborSearchBackend(str, Enum):
    DENSE = "dense"
    CELL_LIST = "cell_list"
    VERLET_CACHE = "verlet_cache"
```

`VERLET_CACHE` is result provenance for `NeighborSearchSession`. It is not a selectable stateless backend.

`CellListOptions`

```
@dataclass(frozen=True, slots=True)
class CellListOptions:
    use_lattice_reduction: bool = True
    max_stencil_candidates: int = 1_000_000
    max_stencil_offsets: int = 250_000
    metric_tolerance: float = 1.0e-12
    coordinate_tolerance: float = 1.0e-12
    reduction_rtol: float = 1.0e-12
    reduction_atol: float = 1.0e-12
```

Field	Type	Constraint	Meaning
`use_lattice_reduction`	`bool`	Boolean.	Permit an equivalent reduced search basis.
`max_stencil_candidates`	`int`	Positive.	Hard cap on raw stencil candidate work.
`max_stencil_offsets`	`int`	Positive.	Hard cap on retained neighbor-bin offsets.
`metric_tolerance`	`float`	Finite and nonnegative.	Conservative metric-box comparison tolerance.
`coordinate_tolerance`	`float`	Finite and nonnegative.	Bin-coordinate boundary tolerance.
`reduction_rtol`	`float`	Finite and nonnegative.	Relative reduction verification tolerance.
`reduction_atol`	`float`	Finite and nonnegative.	Absolute reduction verification tolerance.

A hard-cap violation raises `CellListComplexityError`. Under `NeighborSearchOptions(backend="auto", fallback_to_dense=True)`, the policy falls back to dense for that exact request.

NeighborSearchOptions

```
@dataclass(frozen=True, slots=True)
class NeighborSearchOptions:
    backend: Literal["auto", "dense", "cell_list"] = "auto"
    cache_mode: Literal["auto", "none", "verlet"] = "auto"
    skin: float = 0.5
    deformation_aware: bool = True
    dense_pair_threshold: int = 32_768
    minimum_cache_frames: int = 2
    max_consecutive_zero_reuse_rebuilds: int = 3
    safety_tolerance: float = 1.0e-12
    max_cell_condition_number: float = 1.0e12
    fallback_to_dense: bool = True
    cell_list_options: CellListOptions = CellListOptions()
```

Option fields and constraints:

- `backend: Literal["auto", "dense", "cell_list"]` selects the requested execution policy.
- `cache_mode: Literal["auto", "none", "verlet"]` selects the cross-frame candidate-reuse policy.
- `skin: float` must be positive and finite; it is the candidate-list margin in angstrom.
- `deformation_aware: bool` permits the proven variable-cell validity bound.
- `dense_pair_threshold: int` must be positive and sets the dense-work boundary used by `auto`.
- `minimum_cache_frames: int` must be positive and sets the minimum selected-frame count for cache eligibility.
- `max_consecutive_zero_reuse_rebuilds: int` must be positive and disables a repeatedly unproductive cache after that many completed zero-reuse intervals.
- `safety_tolerance: float` must be finite and satisfy $0 \leq \epsilon < \delta$.
- `max_cell_condition_number: float` must be finite and greater than one.
- `fallback_to_dense: bool` permits automatic recovery from exact cell-list complexity.
- `cell_list_options: CellListOptions` must be a valid options object and controls stateless and rebuild cell-list construction.

Here δ denotes `skin` and ϵ denotes `safety_tolerance`.

Behavioral resolution:

- `cache_mode="auto"` activates Verlet reuse only for an eligible time-ordered trajectory;
- a single selected frame never activates a cache;
- independent ensembles are stateless under `cache_mode="auto"`;
- `cache_mode="verlet"` is an expert override and may attempt safe reuse for a geometrically related ensemble;
- `backend="dense"` resolves to no cache without mutating the requested option;
- `backend="cell_list"` is explicit and does not silently fall back to dense;
- `backend="auto"` may fall back to dense only when `fallback_to_dense=True`;
- any active cache is disabled for the rest of the request after the configured number of consecutive completed intervals with zero reuse.

Methods:

```
options.to_dict() -> dict[str, Any]
options.to_verlet_options() -> VerletCacheOptions
```

`to_dict()` is diagnostic, not a stable persistence format.

VerletCacheOptions

```
@dataclass(frozen=True, slots=True)
class VerletCacheOptions:
    skin: float = 0.5
    safety_tolerance: float = 1.0e-12
    deformation_aware: bool = False
    max_cell_condition_number: float = 1.0e12
    cell_list_options: CellListOptions = CellListOptions()
```

This lower-level session option retains the fixed-cell S2 default. The S4 high-level policy converts its own default to `deformation_aware=True` explicitly.

`NeighborListResult`

```
@dataclass(frozen=True, slots=True)
class NeighborListResult:
    frame_index: int
    center_indices: NDArray[np.int64]
    neighbor_indices: NDArray[np.int64]
    offsets: NDArray[np.int64]
    vectors: NDArray[np.float64]
    distances: NDArray[np.float64]
    image_shifts: NDArray[np.int64]
    cutoff: float
    pair_counting: PairCounting
    backend: NeighborSearchBackend = NeighborSearchBackend.DENSE
```

Field	Shape	Contract
`center_indices`	`(n_centers,)`	Ordered center atoms.
`neighbor_indices`	`(n_pairs,)`	Candidate atom for each retained pair.
`offsets`	`(n_centers + 1,)`	Nondecreasing CSR row offsets, first zero, last `n_pairs`.
`vectors`	`(n_pairs, 3)`	Finite minimum-image Cartesian vectors.
`distances`	`(n_pairs,)`	Norms of `vectors`; every value is strictly below `cutoff`.
`image_shifts`	`(n_pairs, 3)`	Integer original-basis image translations.

Useful properties and methods include:

```
result.n_centers -> int
result.n_pairs -> int
result.coordination_counts -> NDArray[np.int64]
result.row(center_position: int) -> tuple[arrays...]
result.summary() -> dict[str, Any]
```

`VerletPairCache`

`VerletPairCache` is an immutable request-specific candidate cache. Its central fields are:

```
request_digest: str
reference_frame_index: int
selected_atom_indices: NDArray[np.int64]
reference_wrapped_positions: NDArray[np.float64]
reference_fractional_positions: NDArray[np.float64]
reference_cell: NDArray[np.float64]
active_pair_atomic_numbers: NDArray[np.int64]
active_pair_cutoffs: NDArray[np.float64]
center_indices: NDArray[np.int64]
candidate_neighbor_indices: NDArray[np.int64]
candidate_offsets: NDArray[np.int64]
physical_cutoff: float
list_cutoff: float
pair_counting: PairCounting
```

```
skin: float
canonical_schema_version: str = "mdstats.verlet-cache.v2"
```

The cache contains only candidates that were inside the pair-specific list radius at rebuild. Users should inspect, not manually construct, this object. Its arrays are defensive, validated, and read-only.

NeighborCacheIntervalStatistics

```
@dataclass(frozen=True, slots=True)
class NeighborCacheIntervalStatistics:
    request_digest: str
    reference_frame_index: int
    last_frame_index: int
    evaluations: int
    reuse_evaluations: int
    candidate_pairs: int
    minimum_safety_margin: float | None
    minimum_singular_value: float | None
    terminal_rebuild_reason: str | None = None
```

One interval begins at a rebuild and ends immediately before the next rebuild or at the latest evaluated frame. `terminal_rebuild_reason=None` means the interval is still current at snapshot time.

NeighborCacheStatistics

```
@dataclass(frozen=True, slots=True)
class NeighborCacheStatistics:
    evaluations: int
    rebuilds: int
    reuse_evaluations: int
    candidate_pair_evaluations: int
    accepted_pairs: int
    current_candidate_pairs: int
    rebuild_reasons: tuple[tuple[str, int], ...]
    rebuild_intervals: tuple[NeighborCacheIntervalStatistics, ...] = ()
```

Derived diagnostics:

```
statistics.mean_evaluations_per_rebuild -> float
statistics.median_evaluations_per_rebuild -> float
statistics.acceptance_ratio -> float
statistics.minimum_safety_margin -> float | None
statistics.minimum_singular_value -> float | None
statistics.to_dict() -> dict[str, Any]
```

The acceptance ratio is

$$\eta = \frac{N_{\text{accepted}}}{N_{\text{candidate evaluations}}}.$$

A small η is not an accuracy failure, but it can indicate an oversized skin, broad cutoff request, or inefficient bin geometry.

NeighborRequestDiagnostics

Each normalized high-level request records:

```
request digest
estimated dense pair work
policy backend
resolved selected-frame semantics
requested and selected cache modes
cache-resolution reason
runtime cache-disable state and reason
consecutive zero-reuse interval count and configured limit
evaluation count
```

```
actual backend counts
candidate and accepted counts
fallback events
```

Requests are ordered by digest in the final diagnostic snapshot.

`NeighborSearchDiagnostics`

```python
@dataclass(frozen=True, slots=True)
class NeighborSearchDiagnostics:
    options: NeighborSearchOptions
    selected_frame_count: int
    frame_semantics: Literal["single_frame", "trajectory", "ensemble"]
    requests: tuple[NeighborRequestDiagnostics, ...]
    cache_statistics: dict[str, Any] | None
```

`to_dict()` emits schema `mdstats.periodic-neighbor-search.v2` and aggregates:

```
resolved selected-frame semantics
backend requested, policy-selected, and actually used
cache mode requested and selected
cache-resolution reasons
runtime cache-disable state and reasons
zero-reuse rebuild limit
skin
selected frames and evaluations
request digests
candidate and accepted pair counts
candidate efficiency
fallback events
cell-list rebuilds and reuse frames
mean and median frames per rebuild
rebuild reasons
minimum safety margin by rebuild interval
minimum singular value by rebuild interval
full request and cache records
```

These fields are provenance only. They must not affect the observable result.

Function calls

Stateless exact builder

```python
build_neighbor_list(
    collection: AtomisticFrameCollection,
    *,
    frame_index: int,
    center_indices: ArrayLike,
    candidate_neighbor_indices: ArrayLike,
    cutoff: float | PairCutoff,
    pair_counting: PairCounting = PairCounting.DIRECTED,
    backend: NeighborSearchBackend = NeighborSearchBackend.DENSE,
    block_size: int = 256,
    cell_list_options: CellListOptions | None = None,
) -> NeighborListResult
```

Inputs:

- `collection` must be a validated fixed-population `AtomisticFrameCollection`;
- `frame_index` must select one stored frame;
- index arrays must be one-dimensional, unique, in range, and compatible with `pair_counting`;
- `cutoff` must be positive, finite, and inside the unique-image regime;
- `block_size` must be a positive integer;

- `backend` may be only dense or cell list.

The function performs no cross-frame reuse.

Persistent session

```
session = NeighborSearchSession(
    collection: AtomisticFrameCollection,
    options: VerletCacheOptions | None = None,
)
```

Core calls:

```
session.build_neighbor_list(
    *,
    frame_index: int,
    center_indices: ArrayLike,
    candidate_neighbor_indices: ArrayLike,
    cutoff: float | PairCutoff,
    pair_counting: PairCounting = PairCounting.DIRECTED,
) -> NeighborListResult

session.statistics(request_digest: str | None = None) -> NeighborCacheStatistics
session.cache_for_request(request_digest: str) -> VerletPairCache
session.clear() -> None
session.n_caches -> int
session.request_digests -> tuple[str, ...]
```

One session is bound to one collection. It owns one current cache per exact request digest. It is not thread-safe for concurrent mutation.

Consumer APIs

The following public analyses accept the same high-level option:

```
compute_pair_rdf(
    ...,
    neighbor_search_options: NeighborSearchOptions | None = None,
) -> RDFResult

compute_coordination_distribution(
    ...,
    neighbor_search_options: NeighborSearchOptions | None = None,
) -> CoordinationResult

compute_bond_angle_distribution(
    ...,
    neighbor_search_options: NeighborSearchOptions | None = None,
) -> BondAngleResult

compute_atomic_connectivity(
    ...,
    neighbor_search_options: NeighborSearchOptions | None = None,
    verlet_cache_options: VerletCacheOptions | None = None,
) -> AtomicConnectivityResult
```

`verlet_cache_options` is retained only as a compatibility path for atomic connectivity. Passing both option objects is an error. New code should use `neighbor_search_options`.

Each result stores the high-level diagnostic dictionary at:

```
result.metadata["neighbor_search"]
```

Atomic connectivity also retains a compatibility `neighbor_cache` metadata alias when cache statistics exist.

Algorithms and theory

Literature provenance and attribution boundary

The S1 domain decomposition follows the cell-linked-list foundation of Quentrec and Brot [1]. Efficient pair-list construction for arbitrary periodic boxes [4], metric neighbor-list methods for parallelepiped cells [5], and closest-point periodic search [6] provide general-cell prior art. The optional basis reduction is supplied by ASE [8], whose low-dimensional reduction routine follows Nguyen and Stehlé [9].

The S2 buffer follows Verlet [2], and its displacement-triggered automatic rebuild rule follows the update lineage represented by Chialvo and Debenedetti [3]. Cell lists designed for dynamically deforming periodic geometries [7] provide related context for S3.

The attribution boundary is strict:

- the exact active-set metric-box stencil, perpendicular-height policy, deterministic original-basis image recovery, and complexity guards are mdstats-specific S1 adaptations;
- request-keyed immutable caching and diagnostic schemas are mdstats-specific S2 engineering;
- the species-resolved smallest-singular-value deformation margin is an mdstats-specific S3 theorem, not a transcription of references [5] or [7].

Blocked dense oracle

The dense backend evaluates all eligible center-candidate pairs in bounded center blocks. For directed selections, the estimated pair work is

$$W_{\mathrm{dense}} = N_c N_n.$$

For unordered identical selections,

$$W_{\mathrm{dense}} = \frac{N(N-1)}{2}.$$

The dense path applies one exact minimum-image operation and strict cutoff test. It is the permanent scientific oracle for optimized-backend tests.

Exact triclinic cell list

The cell-list backend performs the following steps:

```
validate frame, cell, PBC, selections, and cutoff
optionally construct an equivalent reduced lattice basis
map selected wrapped positions to fractional search coordinates
choose periodic and nonperiodic bin counts
assign atoms to deterministic bins
construct a conservative metric-aware neighbor-bin stencil
enumerate center/candidate bin pairs
apply exact original-geometry MIC and pair cutoff filtering
canonicalize, sort, deduplicate, and build CSR output
convert any reduced-basis image shift back to the original cell basis
```

For fractional difference $\Delta\mathbf{s}$, the squared Cartesian distance is

$$\|\Delta\mathbf{s} H\|^2 = \Delta\mathbf{s}\, G\, \Delta\mathbf{s}^T, \qquad G = HH^T.$$

The stencil does not assume orthogonal cells. For each possible pair of fractional bin boxes, it computes a conservative lower bound under this metric. A bin offset is omitted only when the lower bound is safely outside the maximum list radius.

If lattice reduction is used,

$$H_{\mathrm{r}} = UH, \qquad U \in \mathrm{GL}(3,\mathbb{Z}),$$

with U unimodular. Reduction changes only the search representation. Returned vectors and image shifts remain expressed in the original cell basis.

Candidate-list rebuild

For physical pair cutoff r_{ab} and skin δ, rebuild uses list radius

$$r_{ab}^{\mathrm{list}} = r_{ab} + \delta.$$

The exact cell-list backend builds candidates at this list radius. Every reuse frame recomputes current minimum-image geometry for cached candidates and applies only the physical cutoff r_{ab}.

Fixed-cell cache validity

Let $d_{\max}$ be the largest minimum-image displacement of any selected atom from the rebuild reference. Cache completeness is guaranteed while

$$2d_{\max} < \delta - \epsilon,$$

where ϵ is the numerical safety tolerance. Equality rebuilds.

Deformation-aware cache validity

Let H_0 be the rebuild cell and H_t the current cell. The row-vector affine map is

$$F_t = H_0^{-1} H_t.$$

Let $\sigma_{\min}(F_t)$ be its smallest singular value. Continuous trajectory fractional coordinates define each selected atom's nonaffine displacement:

$$\mathbf{u}_i(t) = [\mathbf{s}_i(t) - \mathbf{s}_i(t_0)] H_t.$$

For species a, define

$$d_a(t) = \max_{i \in a} \|\mathbf{u}_i(t)\|.$$

For every active species pair (a, b), the conservative remaining margin is

$$M_{ab}(t) = \sigma_{\min}(F_t)(r_{ab} + \delta) - r_{ab} - d_a(t) - d_b(t).$$

The cache may be reused only when

$$\min_{(a,b)} M_{ab}(t) > \epsilon.$$

This bound is direction-independent and therefore conservative. Rigid cell rotations have $\sigma_{\min} = 1$ and do not consume affine margin. Compression, shear, and nonaffine motion consume margin and trigger a rebuild before an omitted pair can cross the physical cutoff.

If an independent ensemble lacks continuous fractional unwrapping, a changed cell triggers `fractional_unwrapping_unavailable`; fixed-cell frames may still use the fixed-cell displacement test.

Request identity

A cache is request-specific. The digest includes the normalized center and candidate selections, pair-counting mode, physical cutoff, relevant options, collection identity fields, and schema version. A cache is never silently shared across semantically different requests.

The S4 policy digest additionally includes the complete high-level option mapping, includes resolved frame semantics, and uses schema `mdstats.periodic-neighbor-request.v2`.

Automatic backend policy

The initial production policy is deliberately simple and deterministic. Estimate dense work W as above, then choose

$$\mathrm{backend} = \begin{cases} \mathrm{dense}, & W < W_0, \\ \mathrm{cell\_list}, & W \geq W_0, \end{cases}$$

with default

$$W_0 = 32768.$$

This threshold was selected conservatively from the S4 benchmark across small Na-LTA, replicated Na-LTA, dense salt, mixed interface, skewed cells, and synthetic scaling cases. It is not claimed to be hardware-optimal. Users may set `dense_pair_threshold` or force a backend.

Cache resolution is semantics-aware:

Selected-frame semantics	`cache_mode="auto"`
one selected frame	stateless
time-ordered trajectory	Verlet when the cell-list backend and frame-count requirements are satisfied
independent ensemble	stateless

Explicit `cache_mode="verlet"` bypasses the semantics default, but not geometric validity, unique-image safety, or runtime shutoff. Cache activation therefore requires:

```
policy backend == cell_list
selected_frame_count >= minimum_cache_frames
and either:
    cache_mode == verlet
or:
    cache_mode == auto and resolved semantics == trajectory
request not disabled by an unsafe list radius or repeated zero reuse
```

A cache interval begins at a build and ends at the next rebuild. If an interval contains no successful reuse, the per-request zero-reuse counter increments. Any successful reuse resets the counter. Reaching `max_consecutive_zero_reuse_rebuilds` disables caching for the remainder of that request and returns to a stateless cell list at the physical cutoff.

Fallback rules

The production policy permits only exact fallbacks:

1. If `physical_cutoff + skin` exceeds the unique-image radius, a requested Verlet path falls back to an exact stateless cell list at the physical cutoff. The event is `verlet_list_radius_unsafe_to_stateless`.
2. If an active cache completes the configured number of consecutive zero-reuse intervals, caching is disabled for that request and later frames use an exact stateless cell list. The event is `repeated_zero_reuse_to_stateless`.
3. If cell-list stencil complexity exceeds configured hard limits under `backend="auto"` and `fallback_to_dense=True`, the exact request falls back permanently to dense. The event is `cell_list_complexity_to_dense`.
4. Explicit `backend="cell_list"` does not hide complexity errors by falling back to dense.
5. No approximate backend or silent candidate truncation is allowed.

High-level execution pseudocode

```
function analyze_selected_frames(collection, request, options):
    semantics = resolve_selected_frame_semantics(collection, selected_frames)
    executor = NeighborSearchExecutor(collection, options, selected_frame_count)

    for frame in selected_frames:
        work = estimate_dense_pair_work(request)
        policy_backend = choose_backend(options, work)
        cache_mode, reason = resolve_cache_mode(
            requested=options.cache_mode,
            backend=policy_backend,
            semantics=semantics,
            selected_frame_count=selected_frame_count,
        )

        if policy_backend == dense:
```

```
            neighbors = exact_dense(frame, request)

        else if cache_mode == verlet and cache_not_runtime_disabled(request):
            try:
                neighbors, event = session.evaluate_or_rebuild(frame, request)
                update_zero_reuse_interval_counter(event)
                if zero_reuse_limit_reached(request):
                    disable_cache_for_request(repeated_zero_reuse)
            catch unsafe_list_radius:
                disable_cache_for_request(unsafe_list_radius)
                neighbors = exact_cell_list(frame, physical_request)
            catch cell_list_complexity and options.backend == auto:
                pin_request_to_dense()
                neighbors = exact_dense(frame, request)

        else:
            try:
                neighbors = exact_cell_list(frame, request)
            catch cell_list_complexity and options.backend == auto:
                pin_request_to_dense()
                neighbors = exact_dense(frame, request)

        consumer_accumulates_scientific_observable(neighbors)

    result.metadata["neighbor_search"] = executor.diagnostics().to_dict()
```

Consumer integration rules

RDF

RDF uses backend-neutral pair distances. Histogram counts, shell volumes, normalization, cumulative coordination, frame selection, and minimum detection are unchanged. One analysis-local executor may persist across selected frames.

Coordination distribution

Coordination uses CSR row counts. Integer per-atom/per-frame coordination values must match exactly across backends before any distribution or summary statistic is formed.

Bond-angle distribution

Neighbor vectors for each center are backend-neutral inputs. Central coordination filters and angle construction may issue multiple exact requests; they share one analysis-local executor but retain distinct request digests.

Atomic connectivity

Geometric candidates may come from any exact backend. Atomic connectivity owns hysteretic and reference state. Formation and breaking classification use one candidate-distance pass for the current frame. Cache rebuild logic is not an atomic-connectivity responsibility.

Framework and ring layers

Higher topology consumes atomic connectivity states. It never creates, invalidates, or inspects neighbor caches as part of scientific graph identity.

Input constraints and errors

The subsystem expects:

- fixed atom count and identity across collection frames;
- finite cells and coordinates;
- nondegenerate cells compatible with the PBC mask;
- unique, in-range integer atom selections;
- positive finite cutoffs and skins;

- a unique periodic image inside every physical cutoff;
- a unique periodic image inside the list cutoff when caching;
- continuous fractional coordinates for deformation-aware trajectory reuse;
- a cell condition number below `max_cell_condition_number` for S3 checks.

Important errors include:

Error	Meaning
`UnsafeNeighborCutoffError`	Physical or list radius violates unique-image semantics.
`CellListComplexityError`	Exact stencil exceeds configured hard limits.
`InvalidCellGeometryError`	Cell, deformation, or coordinate geometry is invalid.
`CoincidentAtomsError`	Distinct selected atoms coincide inside the cutoff.
`InvalidVerletCacheOptionsError`	Cache options are malformed.
`IncompatibleVerletCacheError`	Stored cache state violates its schema or invariants.

Consumers may translate low-level cutoff errors to module-specific public errors, but must not weaken the geometry contract.

Edge cases and warnings

Single-frame input

A single selected frame is valid and always stateless, including an explicit `cache_mode="verlet"`, because no reuse opportunity exists. Backend selection still applies. The diagnostic reason is `single_frame_stateless`.

Independent ensembles

Cell lists are fully valid for independent ensembles because they are stateless. Automatic Verlet reuse is not assumed: `cache_mode="auto"` resolves to a fresh dense or cell-list search for every selected configuration. The diagnostic reason is `ensemble_default_stateless`.

An expert may request `cache_mode="verlet"` for a geometrically related fixed-cell ensemble. Exact fixed-cell displacement validity still applies. Changed-cell ensembles rebuild with `fractional_unwrapping_unavailable`; no continuous fractional branch is inferred. If repeated intervals contain no reuse, the runtime policy disables the cache and returns to the physical-cutoff cell list.

Wrapped trajectory coordinates

A trajectory presented as independently wrapped fractional coordinates can hide or invent motion. Deformation-aware validity requires the collection's declared continuous trajectory representation.

Large skin

A large skin can violate the unique-image list radius even when the physical cutoff is safe. S4 falls back to stateless cell list for that request and records the event. A large skin can also reduce candidate efficiency enough to erase the cache benefit.

Very small skin

A small skin is correct but may rebuild nearly every frame. It can be slower than a fresh cell list.

Highly skewed cells

Skew increases stencil complexity. Lattice reduction normally improves the search basis. Hard caps prevent unbounded candidate enumeration. Dense fallback remains available under `auto`.

Ill-conditioned cells

A mathematically invertible but badly conditioned cell can make deformation bounds numerically unreliable. S3/S4 reject cells above the configured condition limit rather than pretending reuse is safe.

Coincident atoms

Distinct selected atoms at effectively zero distance raise an error. They are not silently counted as neighbors because they often indicate malformed input.

Multiple periodic images

The subsystem returns at most one image of each atom pair. It is not appropriate when a cutoff intentionally spans multiple images of the same atom.

Concurrent use

`NeighborSearchSession` is mutable and not thread-safe. Use one session per thread/task or synchronize access externally.

Mutable source arrays

Collections are validated at construction, but users should treat geometry and identity arrays as immutable during an analysis. Mutating a collection behind an existing session invalidates request and cache assumptions.

Benchmark portability

The default automatic threshold is conservative, not universally optimal. Processor, NumPy build, cell geometry, species selections, and cutoff all affect timing. Force a backend for controlled performance studies.

Provenance and audit requirements

A production consumer result must make the following recoverable:

```
requested and selected backend
requested and selected cache mode
resolved selected-frame semantics and cache-resolution reason
runtime cache-disable state, reason, and zero-reuse limit
skin and policy options
request digest(s)
estimated dense pair work
candidate and accepted pair counts
candidate efficiency
rebuild and reuse counts
mean and median evaluations per rebuild
rebuild reasons
minimum safety margin per interval
minimum singular value per interval
fallback events
```

Diagnostic ordering must be deterministic. Repeating one analysis with identical input and options must produce equal diagnostics except for external benchmark timing, which is not stored in scientific result metadata.

Complexity and performance

Let N_c and N_n denote center and candidate populations, N the total selected population, K the exact cell-list candidate evaluations, and C the cached candidate count.

Path	Approximate work per frame	Notes
Dense directed	$O(N_c N_n)$	Permanent oracle.
Dense unordered identical	$O(N^2)$	Triangular pair set.
Cell-list rebuild	$O(N + K)$	Geometry-dependent stencil and occupancy.
Cache reuse	$O(C)$	Exact current MIC and physical cutoff.

The cache is useful only when rebuild cost is amortized over enough reuse frames. Candidate efficiency, rebuild frequency, and measured reuse/rebuild times should be examined together.

Usage examples

Conservative automatic policy

```
from mdstats import NeighborSearchOptions, compute_pair_rdf

result = compute_pair_rdf(
    collection,
    "Na",
    "Cl",
    r_max=6.0,
    neighbor_search_options=NeighborSearchOptions(),
)

print(result.metadata["neighbor_search"])
```

Automatic trajectory versus ensemble behavior

```
options = NeighborSearchOptions(
    backend="auto",
    cache_mode="auto",
)
```

For a multi-frame trajectory, this permits an eligible Verlet cache. For an independent ensemble, it remains stateless. To deliberately probe safe reuse in a related ensemble, use `cache_mode="verlet"` and inspect `cache_disabled_during_run`, `cache_disable_reasons`, and `fallback_events`.

Force the dense oracle

```
options = NeighborSearchOptions(backend="dense")
```

The requested cache mode remains visible in diagnostics, but the resolved mode is `none` because the selected backend is not a cell list.

Force a stateless cell list

```
options = NeighborSearchOptions(
    backend="cell_list",
    cache_mode="none",
)
```

Force deformation-aware trajectory reuse

```
options = NeighborSearchOptions(
    backend="cell_list",
    cache_mode="verlet",
    skin=0.5,
    deformation_aware=True,
)
```

Tune automatic crossover conservatively

```
options = NeighborSearchOptions(
    backend="auto",
    dense_pair_threshold=65_536,
)
```

A larger threshold retains dense search for more requests.

Inspect a low-level session

```
from mdstats import NeighborSearchSession, VerletCacheOptions
```

```
session = NeighborSearchSession(
    collection,
    VerletCacheOptions(skin=0.5, deformation_aware=True),
)

for frame in range(collection.n_frames):
    neighbors = session.build_neighbor_list(
        frame_index=frame,
        center_indices=centers,
        candidate_neighbor_indices=candidates,
        cutoff=4.0,
    )

statistics = session.statistics()
```

Testing and acceptance contract

The optimized subsystem is accepted only when:

1. dense remains an independent exact oracle;
2. cell-list outputs match dense fields exactly or within the stated floating geometry tolerance;
3. cached outputs match a fresh exact calculation on every frame;
4. randomized orthogonal, triclinic, partial-PBC, boundary, and basis-equivalent fixtures pass;
5. omitted-pair adversarial tests cannot cross the physical cutoff while the reported margin is positive;
6. RDF, coordination, bond-angle, distance, hysteretic, and reference connectivity outputs are backend-neutral;
7. automatic selection never returns an unsupported backend;
8. fallback events are explicit and exact;
9. automatic cache resolution distinguishes single-frame, trajectory, and ensemble semantics;
10. three completed zero-reuse intervals disable caching by default, while any successful reuse resets the counter;
11. diagnostics are deterministic;
12. Markdown/PDF specifications, built distributions, and installed-wheel smoke tests agree with source behavior.

Deferred features

The first production implementation does not provide:

- approximate neighbor search;
- multiple images of one atom pair;
- process-wide or cross-analysis cache sharing;
- superset-cache reuse between different requests;
- pair-specific skins;
- automatic geometric probing or clustering of ensembles;
- adaptive online skin optimization;
- GPU, multithreaded, or distributed cell-list construction;
- cache serialization across program runs;
- automatic hardware calibration at import time.

These may be added only if the scientific neighbor contract and explicit provenance remain unchanged.

AI implementation context

For future code work, retain these invariants:

```
Scientific contract: strict MIC distance < cutoff.
Dense is the permanent oracle.
Cell list is exact and may use a reduced search basis only internally.
Returned vectors/shifts always use the original cell basis.
Verlet rebuilds at physical cutoff + skin; reuse filters at physical cutoff.
Automatic cache reuse requires explicit trajectory semantics and multiple selected frames.
Independent ensembles are stateless by default; explicit Verlet is an expert override.
```

```
Repeated zero-reuse intervals disable the cache and restore the physical-cutoff cell list.
Fixed-cell validity: 2*dmax < skin - tolerance.
Variable-cell validity: every species-pair margin is > tolerance.
The consumer owns scientific state; the neighbor subsystem owns geometry/cache.
Auto is deterministic: dense pair work < threshold -> dense, else cell list.
Only exact, reported fallbacks are allowed.
One analysis-local executor shares sessions across that analysis only.
Diagnostics never participate in graph identity or numerical normalization.
```

Before modifying any consumer, compare the final scientific observable under forced dense, forced stateless cell list, and forced Verlet modes. Do not infer correctness from cache digests or graph digests alone.

References

1. Quentrec, B., and Brot, C. (1973). *New Method for Searching for Neighbors in Molecular Dynamics Computations*. Journal of Computational Physics, 13(3), 430-432. DOI: 10.1016/0021-9991(73)90046-6.
2. Verlet, L. (1967). *Computer "Experiments" on Classical Fluids. I. Thermodynamical Properties of Lennard-Jones Molecules*. Physical Review, 159(1), 98-103. DOI: 10.1103/PhysRev.159.98.
3. Chialvo, A. A., and Debenedetti, P. G. (1990). *On the Use of the Verlet Neighbor List in Molecular Dynamics*. Computer Physics Communications, 60(2), 215-224. DOI: 10.1016/0010-4655(90)90007-N.
4. Heinz, T. N., and Hünenberger, P. H. (2004). *A Fast Pairlist-Construction Algorithm for Molecular Simulations under Periodic Boundary Conditions*. Journal of Computational Chemistry, 25(12), 1474-1486. DOI: 10.1002/jcc.20071.
5. Cui, Z., Sun, Y., and Qu, J. (2009). *The Neighbor List Algorithm for a Parallelepiped Box in Molecular Dynamics Simulations*. Chinese Science Bulletin, 54(9), 1463-1469. DOI: 10.1007/s11434-009-0197-0.
6. Rogers, D. M. (2016). *Overcoming the Minimum Image Constraint Using the Closest Point Search*. Journal of Molecular Graphics and Modelling, 68, 197-205. DOI: 10.1016/j.jmgm.2016.07.004.
7. Dobson, M., Fox, I., and Saracino, A. (2016). *Cell List Algorithms for Nonequilibrium Molecular Dynamics*. Journal of Computational Physics, 315, 211-220. DOI: 10.1016/j.jcp.2016.03.056.
8. Larsen, A. H., et al. (2017). *The Atomic Simulation Environment - A Python Library for Working with Atoms*. Journal of Physics: Condensed Matter, 29(27), 273002. DOI: 10.1088/1361-648X/aa680e.
9. Nguyen, P. Q., and Stehlé, D. (2009). *Low-Dimensional Lattice Basis Reduction Revisited*. ACM Transactions on Algorithms, 5(4), Article 46. DOI: 10.1145/1597036.1597050.